A per-sample offset for scikit-learn logistic regression

Moussa Ouallaf
Husimi.ai
moussa@husimi.ai

Technical note (preprint, not peer-reviewed)

Abstract

scikit-learn's `LogisticRegression` cannot take a per-sample offset (a fixed term added to each example's log-odds), so a known reference probability can only enter as a feature. We add `offset=` to `fit` and every prediction method, fix the Newton line search that would count the offset twice, and ship a package that uses a cross-validated fraction κ of the reference's log-odds as the offset; the fit matches statsmodels and glum to 1.2×10^{-8} in the coefficients. On three public datasets the offset does about as well as the reference entered as a nearly unpenalised feature: it is ahead beyond 2 SE in 1 of 8 cells (HIGGS, $n = 1{,}000$: -0.0019, SE 0.0006) and behind in none. What it buys is exactness, one fewer fitted coefficient and a ridge that shrinks toward the reference instead of toward zero, not lower loss. Using the reference at all beats pristine `LogisticRegressionCV` without it on HIGGS (offset: -0.0529, SE 0.0044, at 250 rows) and Bank Marketing (-0.0882, SE 0.0054), but not on the drifting Elec2 stream; the free column gets the same gain. On HIGGS the reference alone beats the offset fit at 250 rows (+0.0034, SE 0.0013). A post-hoc one-SE rule toward $\kappa = 1$ beat the free column under the same rule on fresh draws at $n = 100$ (-0.0103, SE 0.0050), not at $n = 150$, and never beat the reference alone. SEs are between disjoint repeats of paired held-out differences.

1 Problem

An offset is a known term added to the linear score of a generalised linear model (GLM) [McCullagh and Nelder, 1989]. The model learns weights for the features, but the offset keeps its coefficient fixed at one. R's `glm`, glmnet [Friedman et al., 2010], statsmodels [Seabold and Perktold, 2010] and glum [QuantCo, 2026] all accept it. scikit-learn [Pedregosa et al., 2011] does not, and the request to add it is still an open issue on the project's tracker [scikit-learn developers, 2022].

The case we care about is a strong reference: every example already has a probability from an older model, other data or an outside source. A ridge logistic model on a few hundred rows should then learn only the correction that the features add. Without an offset the reference's log-odds must go in as one more column, and the ridge shrinks the coefficient of the strongest input in the model. Scaling that column up makes its coefficient nearly free, which removes the shrinkage but also any pull toward one. The change came out of a private application, and Appendix C gives the one result we report from it. Everything else is measured on public benchmarks.

2 The change

We add an array `offset` of shape $(n,)$ to `fit`, `decision_function`, `predict`, `predict_proba` and `predict_log_proba`, in the same way as `sample_weight`. The model and its objective are

$$\eta_i = x_i^\top w + b + o_i, \qquad \min_{w,b} \; C \sum_{i=1}^{n} s_i \, \ell(y_i, \eta_i) + \tfrac{1}{2}\|w\|^2, \tag{1}$$

Preprint.

where η_i is the log-odds of example i, o_i its given (not fitted) offset, s_i its sample weight, C the inverse ridge strength and $\ell(y, \eta) = \log(1 + e^{\eta}) - y\eta$ the log-loss. The offset is data: it is never stored, and $o =$ `None` gives the current behaviour bit for bit. Our library diff is 232 added and 19 removed lines, with 318 lines of tests. One removed line fixes a bug the offset exposed: the Newton line search evaluates $\eta(w + t\,d) = \eta(w) + t\,Xd$, but built Xd with the helper that adds o, so the offset counted twice per step and newton-cholesky stalled or fell back to lbfgs.

Our package `anchorlogit` builds the offset from a reference probability p_i, with $r_i = \mathrm{logit}(p_i)$:

$$o_i = \kappa\, r_i, \qquad \kappa \in \{0, 0.1, 0.25, 0.5, 0.75, 1\}, \tag{2}$$

where κ is the fraction of the reference we trust, chosen jointly with C by 5-fold cross-validated log-loss. $\kappa = 1$ is the classical GLM offset and $\kappa = 0$ ignores the reference. For training rows the reference must be out of sample: if it was fitted on the same rows, `anchorlogit` cross-fits it (each row gets a prediction from a fit that did not see it) [Chernozhukov et al., 2018].

The recipe needs nothing specific to a domain: (i) For every example, training and new, get a reference probability p_i from a source that did not see that example's label, and clip it away from 0 and 1. (ii) Compute $r_i = \mathrm{logit}(p_i)$. (iii) Fit on the training rows with `offset=` κr for each (κ, C) on the grid, and keep the pair with the lowest cross-validated log-loss. (iv) At prediction time pass `offset=` κr for the new rows as well; the model returns the corrected log-odds. In the package, `AnchoredLogisticRegression(anchor=j, kappa="cv")` takes p as column `j` of X and does (ii) to (iv), with C tuned by any scikit-learn grid search; given an unfitted classifier as `anchor`, it also does (i) by cross-fitting.

A general form lets the coefficient on r move but pulls it toward κ, with an anchoring strength λ:

$$J(w, b, \beta) = C \sum_i s_i\, \ell\big(y_i,\ x_i^{\top} w + b + \beta r_i\big) + \tfrac{1}{2}\|w\|^2 + \tfrac{1}{2}\lambda(\beta - \kappa)^2. \tag{3}$$

$\lambda = \infty$ is the fixed offset of Equation 2 and $\lambda = 0$ is the reference as a free column. Lemma 1 shows that every λ in between needs nothing beyond `offset=`.

Lemma 1. *For $0 < \lambda < \infty$, (w^*, b^*, u^*) minimises the offset problem (1) with offset $o_i = \kappa r_i$ and one extra ridge column $r_i/\sqrt{\lambda}$ (weight u) if and only if $(w^*, b^*, \kappa + u^*/\sqrt{\lambda})$ minimises J.*

The proof is in Appendix A; the check in code matches the direct minimisation of J to 3.3×10^{-9} in η over 18 cases.

3 Setup

We use three public datasets from OpenML [Vanschoren et al., 2014], each with a different kind of reference. **HIGGS** [Baldi et al., 2014]: gradient boosting on the 21 low-level columns, fitted once on 60,000 disjoint rows, is the reference; the ridge sees all 28 columns, so the features partly re-express it. **Bank Marketing** [Moro et al., 2014]: gradient boosting on every column, fitted on a disjoint half, so the features add nothing (the expected null). **Elec2** [Harries, 1999]: a time-ordered stream; the reference is fitted on rows before each origin with an embargo (a gap of left-out rows) of 48 rows, so it ages, and the test set is the next week. Within a repeat every arm sees the same training rows, the same features and the same folds; only the use of r differs. HIGGS has 10 repeats, Bank 8 and Elec2 12; each repeat is a disjoint (train, test) draw with 1,500, 1,200 and 336 test rows.

Table 1: All arms share one cross-validation grid and the same folds, so the tuning budget is equal; a1 is the pristine upstream, a2 the trivial baseline. Platt scaling refits only the reference's slope and level.

Arm	Model	Arm	Model
a1	pristine `LogisticRegressionCV`, no reference	a5	offset, $\kappa = 1$ (classical GLM offset)
a2	reference alone (a2$_{\mathrm{P}}$: Platt [Platt, 1999])	a6	offset, (κ, C) by CV (ours)
a3	pristine, r as a standardised feature	a7	anchored, (κ, λ, C) by CV, $\lambda \in \{\infty, 10, 1, 0.1, 0\}$
a4	same, r times 100 (nearly free, $\lambda \approx 0$)	a9	a6, reference refit in-sample (leakage)
		a10	LightGBM [Ke et al., 2017] started at $\kappa_{\mathrm{cv}}\, r$

C runs over 11 values from 10^{-4} to 10. The metric is test log-loss in nats (natural-log units). For arm a against ours, repeat j of R gives the paired difference over its test rows T_j:

$$d_j = \frac{1}{|T_j|} \sum_{i \in T_j} \big[\ell(y_i, \eta_i^{\text{ours}}) - \ell(y_i, \eta_i^{a})\big], \qquad \bar{d} = \frac{1}{R} \sum_{j=1}^{R} d_j, \qquad \text{SE} = \frac{\text{sd}(d_1, \ldots, d_R)}{\sqrt{R}}. \tag{4}$$

Negative $\bar{d}$ favours ours. Because the repeats are disjoint, this SE includes the noise of the training draw. We call it a win only when $|\bar{d}| > 2\,\text{SE}$, and anything inside that is a tie. We fixed 3 checks before the runs: a6 beats a1 on most of the 3 datasets, a6 versus a3 and a4, and $\kappa_{\text{cv}} < 1$ where the reference is off.

4 Results

In the main runs (Table 2) the offset beats the model without the reference in 6 of 8 cells, all on HIGGS and Bank. The free column a4 is within 0.002 of the offset in each of those cells, so this gap shows that the reference helps, not that the offset does. On HIGGS the gap falls from -0.0529 at $n = 250$ to -0.0276 at $n = 2{,}000$, and on Bank it is -0.0882 at $n = 250$. The reference alone (a2) is harder: on HIGGS it beats the offset at $n = 250$ (+0.0034, SE 0.0013) and at $n = 100$ to 250 on the small-n draws, ties at 500 and 1,000, and loses at 2,000 (-0.0089, SE 0.0013); on Bank and Elec2 it ties.

The key comparison is the reference entered as a column. The penalised column a3 loses to the offset at small n (HIGGS $n = 250$: -0.0185, SE 0.0034; Bank: -0.0591, SE 0.0033), because the ridge shrinks its coefficient; the nearly free a4 does not. Against a4 the offset wins in 1 of 8 cells and loses in 0, and the anchored a7 also wins in 1. The single win is HIGGS at $n = 1{,}000$ (-0.0019, SE 0.0006), and in that cell a3 ties the offset (+0.0009, SE 0.0011).

Table 2: The offset ties the free column and beats the model without the reference on HIGGS and Bank, not on Elec2. Test log-loss per arm; $\bar{d}$ = a6 minus the arm with its between-repeat SE; bold is beyond 2 SE.

Data, n	a1	a2	a4	a5	a6	$\bar{d}$ vs a1	$\bar{d}$ vs a4	$\bar{d}$ vs a5
HIGGS 250	0.6839	0.6275	0.6321	0.6300	0.6309	$-$**0.0529** (0.0044)	-0.0012 (0.0020)	+0.0009 (0.0006)
HIGGS 500	0.6722	0.6275	0.6286	0.6287	0.6291	$-$**0.0431** (0.0046)	+0.0005 (0.0007)	+0.0004 (0.0004)
HIGGS 1,000	0.6536	0.6275	0.6278	0.6257	0.6259	$-$**0.0277** (0.0030)	$-$**0.0019** (0.0006)	+0.0002 (0.0002)
HIGGS 2,000	0.6461	0.6275	0.6187	0.6189	0.6186	$-$**0.0276** (0.0022)	-0.0002 (0.0002)	-0.0003 (0.0006)
Bank 250	0.2920	0.1991	0.2050	0.2051	0.2038	$-$**0.0882** (0.0054)	-0.0012 (0.0008)	-0.0013 (0.0009)
Bank 1,000	0.2548	0.1991	0.1984	0.2000	0.1986	$-$**0.0561** (0.0043)	+0.0002 (0.0005)	-0.0014 (0.0009)
Elec2 250	1.1382	0.9089	1.2465	1.3447	1.2175	+0.0793 (0.0465)	-0.0290 (0.0346)	$-$**0.1272** (0.0349)
Elec2 1,000	0.6864	0.9089	0.6940	0.8505	0.6926	+0.0062 (0.0098)	-0.0014 (0.0031)	$-$**0.1580** (0.0372)

In these runs the fraction κ mattered only where the reference was miscalibrated. On HIGGS cross-validation picks $\kappa = 1$ in 8/10 draws at $n = 250$, and a6 ties a5. On Elec2 the aged reference gets a median κ of 0.25 at $n = 1{,}000$, and a6 beats the full offset by -0.1580 (SE 0.0372); Figure 1 shows every comparison.

To find what would make the offset win, we tried 3 post-hoc variants and kept only the first. At $n = 100$ the CV minimum over κ was noisy, and κ_{cv} was worse than $\kappa = 1$ by +0.0044 (SE 0.0023), just inside 2 SE. The one-SE rule [Hastie et al., 2009] takes, among all (κ, C) within one fold-SE of the best cross-validated loss, the largest κ and then the strongest ridge:

$$(\hat{\kappa}, \hat{C}) = \underset{(\kappa, C) \in S}{\arg\max}\,(\kappa, -C), \qquad S = \{(\kappa, C) : \text{CV}(\kappa, C) \le \text{CV}_{\min} + \text{SE}_{\min}\}, \tag{5}$$

where the arg max is lexicographic (compare κ first, then C) and $\text{SE}_{\min}$ is the fold SE at the CV minimum. To keep the comparison fair, the free column gets the same C rule as a control (Figure 1(d), Table 5).

On the draws it was chosen on, the rule beats the free column under its own C rule at $n = 150$ (-0.0064, SE 0.0030), and at $n = 100$ it is just inside 2 SE (-0.0082, SE 0.0044). Chosen after seeing those numbers, it was rerun on 5 fresh HIGGS draws that no variant run used. There the order flips: a win at $n = 100$ (-0.0103, SE 0.0050) and a tie at $n = 150$ (-0.0021, SE 0.0016). It loses

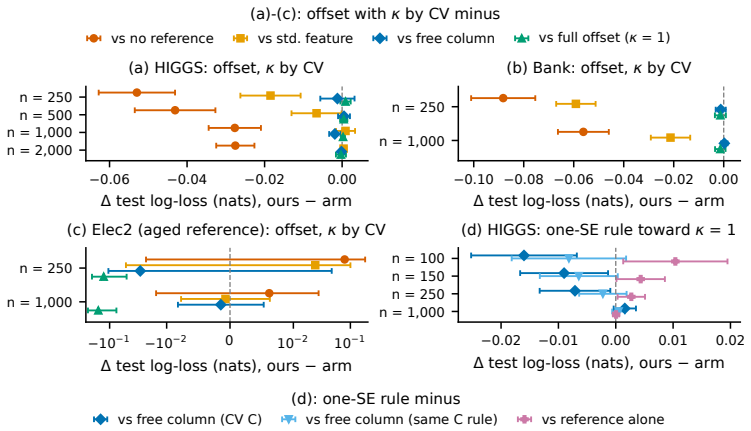


Figure 1: The offset beats no reference on HIGGS and Bank and the penalised column at small n, and sits on zero against the free column. Paired test log-loss difference with 95% intervals from the between-repeat SE; (c) has a symmetric log axis; (d) uses the draws the rule was selected on (fresh draws in Table 5).

to the reference alone at every $n \leq 250$ tested, on the fresh draws ($n = 100$ and 150) too. It selects $\kappa = 1$ in every draw: the classical offset with a conservative ridge.

A reference built from all 28 HIGGS columns, which the features only re-express, gave a tie with the free column (+0.0003, SE 0.0027, $n = 100$) and a loss to the reference alone. A Bank reference trained only on younger clients, wrong in a group the features can see, is where the fraction pays: κ_{cv} has median 0.75 and beats $\kappa = 1$ by -0.0077 (SE 0.0012) at $n = 1{,}000$. It still ties the free column (-0.0004, SE 0.0003) and the Platt-recalibrated reference.

In Figure 2 the expected calibration error (ECE, mean gap between predicted and observed frequency) of the offset and the free column are close; Appendix B has the ablations and checks.

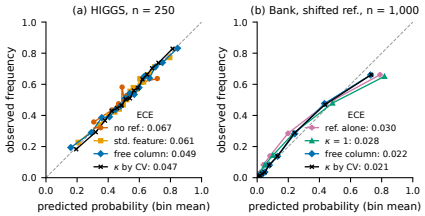


Figure 2: The offset is about as well calibrated as the free column. Reliability on pooled test rows, 15 equal-mass bins; legend ECE is the mean over repeats.

5 Limits

On HIGGS the offset fit loses to the reference alone at $n \leq 250$ and beats it only at 2,000: with few rows, check the reference alone first. The headline is a tie: the offset beats the nearly free column in 1 of 8 cells (1 under a two-stage bootstrap SE that resamples repeats, then test rows), so scaling one column up does about as well; the offset's value is exactness, one fewer coefficient and its shrinkage target, not lower loss.

On Elec2 the offset does not beat a1 (+0.0793, SE 0.0465), and the Platt-recalibrated reference reaches 0.6347 against its 1.2175 at $n = 250$ (a tie only by the large SE 0.2918): $\kappa < 1$ is a coarse fix for a reference wrong in slope and level. Of the 3 pre-registered checks, the win over a1 held on two of three datasets as required (not Elec2); a6 beat a3 on Bank and on HIGGS at $n \leq 500$, tied it elsewhere, and tied a4; $\kappa_{\mathrm{cv}} < 1$ held on Elec2 (ACS Income not run). The evidence is three datasets, gradient-boosting references and $n \leq 2{,}000$ (larger n and more repeats were cut for budget); Elec2 windows share a reference history, so their SE is optimistic. Binary only: the Poisson, Gamma and Tweedie regressors have only the patch's tests.

Generative AI acknowledgement

Generative AI is becoming part of how research is designed, carried out and written up. Readers should know where it was used, so we state it plainly. Generative AI was used throughout this project: to write and review code, to run and check the experiments, and to draft and revise this note. We used only Anthropic's Claude models. Every result reported here comes from code in the accompanying repository and can be reproduced from it.

Statements like this one are still free text, and no two papers word them alike. We are developing a general checklist for reporting generative AI use, in the spirit of the CRediT taxonomy [Brand et al., 2015], which lists each author's contribution through a fixed set of roles. It will cover the ways AI enters a project, from coding assistance and brainstorming to fully autonomous execution, so that researchers can state their use in a short, comparable form. Once published, the checklist will be added to this section.

References

Pierre Baldi, Peter Sadowski, and Daniel Whiteson. Searching for exotic particles in high-energy physics with deep learning. *Nature Communications*, 5:4308, 2014. doi: 10.1038/ncomms5308.

Amy Brand, Liz Allen, Micah Altman, Marjorie Hlava, and Jo Scott. Beyond authorship: attribution, contribution, collaboration, and credit. *Learned Publishing*, 28(2):151–155, 2015. doi: 10.1087/20150211.

Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018. doi: 10.1111/ectj.12097.

Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010. doi: 10.18637/jss.v033.i01.

Michael Harries. Splice-2 comparative evaluation: Electricity pricing. Technical report, University of New South Wales, School of Computer Science and Engineering, 1999.

Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2 edition, 2009. doi: 10.1007/978-0-387-84858-7.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154, 2017. URL `https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html`.

Peter McCullagh and John A. Nelder. *Generalized Linear Models*. Chapman and Hall, London, 2 edition, 1989. doi: 10.1007/978-1-4899-3242-6.

Sérgio Moro, Paulo Cortez, and Paulo Rita. A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems*, 62:22–31, 2014. doi: 10.1016/j.dss.2014.03.001.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL `https://jmlr.org/papers/v12/pedregosa11a.html`.

John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 1999.

QuantCo. glum: High performance Python GLMs with all the features. `https://github.com/Quantco/glum`, 2026. Software, version 3.4.1.

scikit-learn developers. GLM doesn't have an offset option (issue #24614). `https://github.com/scikit-learn/scikit-learn/issues/24614`, 2022. GitHub issue, opened October 2022, accessed 2026-11-03.

Skipper Seabold and Josef Perktold. Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010. doi: 10.25080/Majora-92bf1922-011.

Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. OpenML: Networked science in machine learning. *SIGKDD Explorations*, 15(2):49–60, 2014. doi: 10.1145/2641190.2641198.

A Proof of Lemma 1

Proof of Lemma 1. Given:

1. J as in (3);
2. the offset problem with the extra column, $K(w, b, u) = C \sum_i s_i \, \ell\big(y_i,\, x_i^\top w + b + u\, r_i/\sqrt{\lambda} + \kappa r_i\big) + \frac{1}{2}\|w\|^2 + \frac{1}{2}u^2$;
3. if ϕ is a bijection and $K(v) = J(\phi(v))$ for all v, then v^* minimises K if and only if $\phi(v^*)$ minimises J.

Choose: $\phi(w, b, u) = (w,\ b,\ \kappa + u/\sqrt{\lambda})$, so $\beta = \kappa + u/\sqrt{\lambda}$.

Need to prove: $J(w, b, \kappa + u/\sqrt{\lambda}) = K(w, b, u)$ for all (w, b, u). The ridge term $\frac{1}{2}\|w\|^2$ is the same in both. The score term:

$$\begin{aligned} \beta r_i &= \big(\kappa + u/\sqrt{\lambda}\big) r_i && \text{substitution of } \beta \\ &= \kappa r_i + u\,\big(r_i/\sqrt{\lambda}\big) && \text{distributive law} \end{aligned}$$

The anchoring term:

$$\begin{aligned} \tfrac{1}{2}\lambda(\beta - \kappa)^2 &= \tfrac{1}{2}\lambda\big(u/\sqrt{\lambda}\big)^2 && \text{substitution of } \beta \\ &= \tfrac{1}{2}\lambda\, u^2/\lambda && \text{power of a quotient} \\ &= \tfrac{1}{2}u^2 && \text{cancellation, } \lambda \neq 0 \end{aligned}$$

Substitution of both pieces into J, with the same ridge term, gives:

$$\begin{aligned} J(w, b, \kappa + u/\sqrt{\lambda}) &= C \sum_i s_i \, \ell\big(y_i,\, x_i^\top w + b + u\, r_i/\sqrt{\lambda} + \kappa r_i\big) && \text{both pieces} \\ &\quad + \tfrac{1}{2}\|w\|^2 + \tfrac{1}{2}u^2 \\ &= K(w, b, u) && \text{definition of } K \end{aligned}$$

Precondition: rule 3 is only valid if ϕ is a bijection, therefore its inverse must exist on all of $\mathbb{R}$:

$$\begin{aligned} \beta = \kappa + u/\sqrt{\lambda} &\iff \beta - \kappa = u/\sqrt{\lambda} && \text{subtract } \kappa \\ &\iff u = \sqrt{\lambda}\,(\beta - \kappa) && \text{multiply by } \sqrt{\lambda},\ 0 < \lambda < \infty \end{aligned}$$

Every $\beta \in \mathbb{R}$ has exactly one such u, and w and b map to themselves, so ϕ is a bijection.

Therefore, by rule 3: (w^*, b^*, u^*) minimises K if and only if $(w^*, b^*, \kappa + u^*/\sqrt{\lambda})$ minimises J. □

B Ablations and checks

Table 3 changes one part of the method at a time on HIGGS. Fixing $\kappa = 1$ (a5) or adding the anchoring strength (a7) changes nothing beyond 2 SE. Dropping from the ridge the 21 columns the reference was built from (a8) helps at $n \geq 500$: those columns only re-express the reference and cost variance. A reference refit on the training rows must be cross-fitted: refit in-sample it leaks the labels,

Table 3: On HIGGS fixing $\kappa = 1$ or adding the anchoring changes nothing; dropping the reference's source columns, the source of the reference and the model class do. $\bar{d}$ = a6 minus the arm (SE), 10 repeats; positive means the arm is better; bold is beyond 2 SE.

n	a5: $\kappa = 1$	a7: anchored	a8: no source columns	a9: refit, cross-fitted	a9: refit, in-sample	a10: LightGBM
250	+0.0009 (0.0006)	−0.0020 (0.0021)	+0.0030 (0.0026)	**−0.0535** (0.0040)	**−0.5133** (0.0358)	**−0.0119** (0.0052)
500	+0.0004 (0.0004)	+0.0002 (0.0010)	**+0.0062** (0.0019)	**−0.0431** (0.0046)	**−0.3737** (0.0218)	**+0.0210** (0.0021)
1,000	+0.0002 (0.0002)	−0.0005 (0.0003)	**+0.0089** (0.0014)	**−0.0277** (0.0030)	**−0.1717** (0.0044)	**+0.0387** (0.0026)
2,000	−0.0003 (0.0006)	+0.0000 (0.0000)	**+0.0049** (0.0016)	**−0.0259** (0.0022)	**−0.0827** (0.0032)	**+0.0473** (0.0032)

and at $n = 250$ its log-loss is 1.1443 against 0.6844 cross-fitted. A nonlinear correction beats every linear arm once $n \geq 500$: LightGBM started at the same offset (a10) is better than a6 by +0.0387 (SE 0.0026) at $n = 1{,}000$.

The implementation itself is exact (Table 4). 61 new upstream tests pass on scikit-learn 1.9.1, and the package passes 295 tests with 0 failures.

Table 4: The patched fit matches the reference GLM implementations and stock scikit-learn to solver tolerance. Largest absolute difference over the cases.

Check	cases	coefficients	log-odds η
patched fit vs statsmodels GLM with offset	12	8.3×10^{-9}	3.6×10^{-8}
patched fit vs glum with offset	18	1.2×10^{-8}	4.4×10^{-8}
zero offset vs stock scikit-learn	9	1.9×10^{-8}	not stored
Lemma 1: offset + column vs direct minimisation of J (coefficient: β)	18	2.5×10^{-10}	3.3×10^{-9}

Table 5 lists the one-SE rule of Section 4 against its controls.

Table 5: The one-SE rule beats the free column at small n by about 2 SE, not stably across draws, and never beats the reference alone. $\bar{d}$ = rule minus arm (SE), HIGGS, 10 repeats (top) and 5 fresh repeats (bottom); bold is beyond 2 SE; the rule picks $\kappa = 1$ in every draw.

n	vs a4 (CV C)	vs free column, same rule	vs reference alone	$\kappa = 1$ picks
100	**−0.0160** (0.0041)	−0.0082 (0.0044)	**+0.0104** (0.0040)	10/10
150	**−0.0090** (0.0034)	**−0.0064** (0.0030)	**+0.0043** (0.0019)	10/10
250	**−0.0071** (0.0027)	−0.0022 (0.0018)	**+0.0027** (0.0011)	10/10
1,000	+0.0016 (0.0009)	+0.0005 (0.0005)	+0.0001 (0.0002)	10/10
Fresh draws, 5 repeats, after selection				
100	−0.0090 (0.0045)	**−0.0103** (0.0050)	**+0.0149** (0.0072)	5/5
150	−0.0121 (0.0068)	−0.0021 (0.0016)	**+0.0055** (0.0018)	5/5

C Origin

On a proprietary binary forecasting task with a strong external reference probability and a few hundred training examples, refitting a ridge logistic model with a fixed offset of 0.25 x the reference's log-odds lowered held-out log-loss on about a thousand examples from 0.320 to 0.306 relative to the same model without the offset (paired difference -0.014, moving-block bootstrap SE 0.0035). The fraction was not the best choice: the full offset ($\kappa = 1$) reached 0.283 (0.25 minus 1: +0.023, SE 0.005) and the reference log-odds as an ordinary, nearly unpenalised feature reached 0.287 (offset 0.25 minus feature: +0.018, SE 0.006), the full offset and the free feature being within 0.004 (SE 0.002) of each other.

D Reproduction

All code and results are at commit `6a6daad` of the `anchorlogit` repository. The patch is `patches/scikit-learn-offset.patch` (its full diff against scikit-learn 1.9.1); the benchmarks run with `bench/run_all.sh` and `bench/run_variants.sh`, and `python -m bench.summarize` prints every table. The fresh-draw check of the one-SE rule is `python paper/results/confirm_1se.py`. Numbers in this note come from

`paper/results/make_numbers.py` and `make_extra.py`; recomputing the means and SEs from the stored per-row test log-odds differs from the result files by at most 2.3×10^{-9}. All runs used CPU only with 2 threads per process, 3.5 h in total (HIGGS 63 min, Elec2 38, Bank 31), Python 3.12.3, scikit-learn 1.9.1, statsmodels 0.15.0, glum 3.4.1, `anchorlogit` 0.1.0. On stock scikit-learn without the patch the package passes 288 tests and skips 10.